Конструирование и тестирование ПО

	Балл	
Л.р. 1	15	Scala
Л.р. 2	15	Kotlin
Л.р. 3	20	Android
Л.р. 4	10	Тестирование
Экзамен	40	

Исходные данные. Результаты Л.р. ТРПО-1. Разработка класса структуры данных

Портировать одну из структур данных (СД), приведенных в ermak.cs.nstu.ru/cprog. СД хранит данные произвольного типа:

- Количество хранимых типов – не менее 2, обертка для 1-3 и 4-12
- Экземпляр СД хранит объекты только одного типа (например, только целые или только строки)

Набор операций для структуры данных:

- добавление в конец, получение, вставка и удаление по логическому номеру (индексу). Логический номер – порядковый номер хранимого элемента в соответствии с последовательностью обхода структуры данных (перебора элементов)
- для исходно упорядоченной (например, «двоичное дерево») вставка с сохранением порядка, получение и удаление по по логическому номеру (индексу)
- Итератор forEach – для Java: интерфейс обратного вызова с методом void toDo(T v), итератора получает ссылку на интерфейсный объект (анонимный класс), обходит структуру данных и вызывает метод для каждого выбранного элемента. В Scala – формальный параметр = функция.
- сортировка, для исходно упорядоченной – балансировка = создание экземпляра структуры данных с равными размерностями компонент (например, сбалансированное дерево, либо список списков со списками нижнего уровня равной длины)
- сериализация хранимых данных в текстовый (двоичный) файл в формате по выбору – XML, JSON, собственный двоичный (сохранение и загрузка)

Структуру данных реализовать в виде шаблона или реализации интерфейса, работающего с произвольным типом данных (ТД).

Варианты заданий

1. Двоичное дерево
2. Двоичное дерево в массиве
3. Дерево, упорядоченное вертикально

4. Дерево, упорядоченное вертикально в массиве (пирамида)
5. Односвязный список
6. Двусвязный разомкнутый список
7. Циклический список
8. Список с массивом ссылок в каждом элементе
9. Список, содержащий заголовки списков.
10. Вектор, содержащий заголовки списков.
11. Вектор, содержащий заголовки списков (Хэш-таблица)
12. Дерево с распределением по разрядам ключа
13. Дерево, вершина содержит упорядоченный вектор значений ограниченной размерности (B-дерево)
14. Индексированный список – вектор со ссылками на каждый N-ый элемент списка

Типы хранимых данных

Тестовые (дополнительные):

1. Целое
2. Вещественное
3. Строка

Основные:

1. 2D-точка, сравнение по расстоянию от начала координат
2. Дата – операции с компонентами, добавление/вычитание целого числа дней
3. Время (час/мин/сек) - операции с компонентами, сложение, вычитание,
4. GPS-координата с отметкой времени получения, операции с компонентами, расстояние с учетом широты места, сравнение по широте
5. Правильная дробь, представленная целой частью, числителем и знаменателем
6. Целое положительное число, представленное в виде массива его простых множителей (произведение которых дает это число).
7. Вектор на плоскости, представленный в полярной системе координат (длина, угол поворота).
8. Вектор на плоскости, представленный в прямоугольной системе координат
9. Разреженная матрица переменной размерности, ненулевые коэффициенты представлены вектором с элементами x,y,v - координаты, значение – операции над элементами, сложение матриц,
10. Множество, элементами которого являются целые числа. Представлено битовым массивом, i-ый разряд 1/0 – наличие/отсутствие числа в множестве. Операции объединения и пересечения множеств, добавления элемента, проверки на вхождение, разности множеств. Сравнение по кол-ву элементов множества.
11. Целое без знака неограниченной размерности, представленное строкой цифр. Операции – сложение, вычитание, умножение.
    (варианты 9-11) можно делать совместно 2-мя бригадами.

Разработать оконное приложение, работающее со структурой данных. Функции: отображение состояния (содержимого), все операции, сохранение и загрузка из текстового (двоичного) файла.

Замечания по реализации л.р.1.

Обеспечить возможность настройки приложения на работу с выбранным из списка типов данных (как минимум, 2).

Требуется, чтобы для добавления нового ТД нужно было бы создать класс ТД, присоединяющий общий для ТД интерфейс и добавить экземпляр этого класса в фабрику. Никакие другие изменения в код **не допускаются.**

Предлагаемый способ реализации

1. Шаблоны в Java не настоящие (generic), можно обойтись без них.
2. Вьюшка должна быть отделена от СД, а СД от хранимого типа (интерфейс ТД)
3. Для хранимых типов - интерфейс, который делает с ним, что надо

```
public interface Comparator{
        public int compare(Object o1, Object o2);
        }
public interface UserType{
        public String typeName();                          // Имя типа
        public Object create();                            // Создает объект ИЛИ
        public Object clone();                             // Клонирует текущий
        public Object readValue(InputStreamReader in); // Создает и читает объект
        public Object parseValue(String ss);               // Создает и парсит содержимое из строки
        public Comparator getTypeComparator;               // Возвращает компаратор для сравнения
        }
```

4. Фабрика возвращает вектор имен типов , на основе которых строится выпадающий список для выбора текущего ТД во вьюшке (если GUI). По имени выбранного ТД фабрика возвращает **объект-прототип.**

```
 public class UserFactory{
        public ArrayList<String> getTypeNameList(){...}
        public UserType getBuilderByName(String name){....}
```

5. Вьюшка получает выбранный по имени UserType и создает список (с неразличимыми хранимыми типами):

- для операций добавления вьющка получает от **объекта-прототипа** клон с данными, пропарсеными из строки или потока
- для сортировки вьюшка получает от **объекта-прототипа** компаратор и передает его классу СД для сортировки
- класс СД создает вектор (ArrayList) ссылок на хранимые объекты (например, для создания выпадающего списка)
- для любых функционально тривиальных операций над элементами списка СД поддерживает лямбды вида

```
 public interface DoWith{
                public void doWith(Object obj);
                }
```

```
public interface TestIt{
        public boolean TestIt(Object obj);
        }
```

Под итераторами forEach и firstThat понимаются методы, которые проводят СД линейно и вызывают переданную лямбду для каждого хранимого элемента данных или проверят его, а если лямбда срабатывает, то возвращают.

```
list.forEach(new DoIt(){  public doIt(Object obj){  System.out.println(obj); }});
list.forEach((a)->{  System.out.println(a); });
```

6. Для выпадающего списка можно создать элемент View на основе базового View выпадающего списка, получающего ArrayList<UserType> для заполнения и CallBack-метод для события – выбора элемента, возвращающего индекс выбранного в списке

7. Варианты сортировки (один из):

- создает новый объект-СД со ссылками на объекты из исходной СД
- упорядочивает ссылки в самой СД
- создает копию СД с копиями - клонами хранимых объектов (две СД разделяют данные) (полностью независимый список с оригинальными данными - это ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ), но тогда нужно в Bulder добавит Object clone (Object proto)

9. main должен демонстрировать один и тот же набор операций (тест) объекта-СД с разными UserTYpe, взятыми из фабрики или реализованными просто в виде разных классов (анонимных)

10. Сортировка должна быть НЕ ТУПОЙ (T=N^2), а умной (T=Nlog2N) см. cprog

11. Отчет - диаграмма классов, ключевые моменты, интерфейсы классов, цитаты нетривиального кода (сортировка, например) см. Метод.материалы - ПИ. Рабочие материалы - Руководство программиста.

Л.р.1. Scala. Разработка класса структуры данных

Портировать из Java разработанный в л.р.1 программный код. Сортировку реализовать с использованием техники **функционального программирования и в императивном стиле**. Алгоритм сортировки:

- Рекурсивное разделение
- Рекурсивное слияние
- Циклическое слияние
- Однократное слияние
- Лексикографическая сортировка
- Способы реализации операции сравнения в шаблоне:
- переопределенная операция сравнения в шаблоне
- параметр метода сортировки – внешняя функция сравнения

Разработать оконное приложение, работающее со структурой данных. Функции: отображение состояния (содержимого), все операции, сохранение и загрузка из текстового (двоичного) файла. Варианты реализации:

- Оконные классы в Scala – ScalaFX
- «ручная» генерация разметки в Scala с использованием классов Java (awt или javax)
- Оконные классы в Java, передача событий из класса Scala в Java через интерфейс обратного вызова (CallBack)

В отчете – особенности специфики реализации тех или иных компонент портируемого кода и его цитаты.

Л.р.2. Kotlin. Разработка класса структуры данных

Портировать из Java разработанный в л.р.1 программный код.

В отчете – особенности специфики реализации тех или иных компонент портируемого кода и его цитаты.

Л.р.3. Вариант 1. Android. Разработка приложения с динамической разметкой

Разработать графический интерфейс с программируемой разметкой: переменное количество элементов, их расположение и перемещение, регулярная обработка событий.

Варианты заданий

Электронная таблица типа Excel с формулами – арифметические действия со значениями ячеек и константами.

Турнирная таблица - симметричное заполнение, определение суммы очков и места.

Игровой автомат с прокруткой картинок.

Редактор разметки для кнопок: создание, установка начального текста и иконки, перетаскивание по форме, изменение размеров.

Кроссворд. XML-файл описания вопросов, ответов и размещения слов.

Игра «Домино» - выставление костей в цепочку, кость – пара кнопок с иконками или изображение.

«Рулетка» - вращающееся колесо числами или иконками. При остановке выводится выпадающее число или иконка. При отсутствии средств поворота изображения перемещать горизонтально расположенное текстовое поле с числом или иконку по дуге окружности.

Аниматор сортировки: элементы сортировки – текстовые поля с числами. Графически отображается процесс перестановки элементов в массиве или перемещения из массива в массив

Л.р.3. . Вариант 2. Android. Портировование кода л.р.1

Л.р.4. Модульное тестирование. Разработка тестового покрытия. Тестирование производительности

Разработка тестового покрытия для модульных тестов

Разработать модульные тесты для заданного метода сортировки, реализованного в л.р. 1-3

Последовательность выполнения:

- **Структурное тестирование**: разработать тестовое покрытие для тестирования операторов, условий и путей
- Написать программу тестирования в JUnit, исполняющую весь тестовый набор, фиксирующую нормальное и ошибочное выполнение каждого теста и выводящую несовпадения
- Внести 2-3 дефекта и проверить тестовое покрытие
- **Функциональное тестирование**: определить тестируемые функциональные свойства и их граничные значения на основе свойств данных, функционального назначения алгоритма и возможных ошибок программирования. Построить тестовый набор
- Написать программу тестирования в JUnit, исполняющую весь тестовый набор, фиксирующую нормальное и ошибочное выполнение каждого теста и выводящую несовпадения
- Внести 2-3 дефекта и проверить тестовое покрытие
- **Тестирование полупрозрачного ящика.** Определить дополнительные тестируемые свойства на основе анализа (инспекции) программы. Построить тестовый набор. Дополнить функциональные тесты.
- Внести дефекты, связанные с особенностями реализации и проверить их обнаружение тестами

Модульные тесты должны проверять корректность исполнения для перечисленных тестовых случаев

- исходный набор содержит одинаковые значения
- исходный набор неупорядочен
- исходный набор упорядочен в прямом порядке
- исходный набор упорядочен в обратном порядке
- в наборе имеются повторяющиеся элементы
- в наборе имеются несколько групп повторяющихся элементов
- экстремальное значение находится в середине набора
- экстремальное значение находится в начале набора
- экстремальное значение находится в конце набора

- в наборе несколько совпадающих экстремальных значений

Тестирование производительности и профилирование памяти метода сортировки

1. В соответствии с документом «**Оценка производительности программ»** выполнить тестирование производительности метода сортировки. Оценить вид зависимости трудоемкости и «грязного» времени работы метода от размерности СД
2. При помощи программных счетчиков или метода, возвращающего размер кучи, определить количество объектов данных и вспомогательных (элементов списка, вершин дерева). Оценить вид зависимости используемого объема памяти от размерности СД